Ali Alemami

Amman, Jordan | ali.alemami01@gmail.com | +962 780 020 203 | github.com/ali-alemami | ali-alemami.github.io | linkedin.com/in/ali-alemami

Computer Science student specializing in backend & low-level systems engineering, with hands-on experience in **C/C++** (POSIX system APIs, AST parsing, multithreading, concurrency, memory models) and **C#/.NET** enterprise software architecture (3-tier design, ADO.NET, SQL Server). Seeking an entry-level backend, systems, or software engineering role.

EDUCATION

42 Amman — Common Core, 22/24 projects complete *(peer-reviewed, project-based)* 2025 – present

The Hashemite University — BSc in Computer Science 2022 – present

TECHNICAL SKILLS

Programming:	C (POSIX/C99), C++ (98/11/17), C# (.NET), SQL (T-SQL), Bash
Systems & Architecture:	Unix process control, AST parsing, non-blocking epoll sockets, multithreading (pthreads), mutexes, 3-tier architecture, IPC
CS Fundamentals:	OOP & Design Patterns, Data Structures & Algorithms, Memory Management (Valgrind/ASan), Computer Networks
Tools & Database:	SQL Server, ADO.NET, Docker, Docker Compose, NGINX, Git, Linux, GDB, Valgrind, Makefiles
Languages (Spoken):	Arabic (Native), English (Fluent / Professional)

42 CORE CURRICULUM — 22/24 PROJECTS COMPLETE

- **Multithreading & Concurrency (C — philo):** Engineered a race-free simulation of Dijkstra's Dining Philosophers using POSIX threads (pthreads) and mutex locks; built dedicated asynchronous monitoring routines and microsecond-precision delta timers (gettimeofday) preventing thread starvation and deadlocks.
- **HTTP Web Server (C++98 — webserv, in progress):** Engineering a high-performance non-blocking HTTP/1.1 server using epoll I/O event multiplexing; implementing stateful request stream parsing (GET/POST/DELETE), NGINX-style configuration parsing, client timeout handling, and CGI execution pipelines.
- **DevOps & Container Infrastructure (inception):** Architected a multi-container infrastructure with Docker Compose on Alpine Linux; configured isolated containers for NGINX with TLS/SSL encryption (v1.2/v1.3), WordPress (PHP-FPM), and MariaDB with dedicated bind mounts, internal bridge networking, and PID 1 entrypoint signal management.
- **Unix Shell & AST Engine (C — minishell):** Implemented a POSIX-compliant shell with an Abstract Syntax Tree (AST) parser to enforce operator precedence (|, &&, ||), nested subshell forks, quote-aware tokenization, heredocs with delimiter expansion, variable expansion, and signal handling; zero fd/memory leaks via Valgrind.
- **3D Graphics & Raytracing Engine (C — mini_rt):** Built a 3D raytracing renderer from scratch in C; implemented vector math (dot/cross products, normalization), quadratic discriminant ray-surface intersections (spheres, planes, cylinders), and Phong reflection models (ambient, diffuse, specular highlights, and hard shadows) rendered via MiniLibX.
- **C++ Modules & Ford-Johnson Algorithm (C++ — cpps):** Completed 42 C++ Modules 00–09 (OOP, Orthodox Canonical Form, polymorphism, templates, STL); implemented the **Ford-Johnson Merge-Insert Sort (CPP09)** optimizing comparison steps using dual STL sequence containers (std::vector, std::deque) and Jacobsthal sequence pairing heuristics.
- **Foundations & Algorithms:** push_swap (bitwise radix sort across two stacks), pipex (multi-pipe fork/dup2), fract-ol (Mandelbrot/Julia sets), minitalk (UNIX signal bit-shift IPC), libft, ft_printf, get_next_line. *In progress:* transcendence (real-time multiplayer web app).

PROGRAMMINGADVICES ROADMAP — 17/24 COURSES COMPLETE

Dr. Mohammed Abu-Hadhoud curriculum — software architecture, advanced problem solving, database engineering, and .NET

- **Bank & ATM Management Engine (C++ — Bank-and-ATM-System-CPP):** Engineered a modular, dual-engine banking & ATM system in C++; implemented bitmask-based user role permissions (bitwise flags), custom delimiter-based file serialization (#//#) for data persistence, and transaction audit logs.
- **Contacts 3-Tier Enterprise Application (C#, ADO.NET, SQL Server — Contacts-App-3Tier):** Architected an enterprise application with strict separation of concerns across Presentation, Business Logic, and Data Access layers; used parameterized T-SQL commands to enforce data integrity and prevent SQL injection.
- **Relational Database Schemas & SQL Engineering (SQL Server — Programming-Advices-Core):** Designed 5 relational database schemas (Medical Clinic Booking, E-Commerce Store, HR/Payroll, Vehicle Registry) with primary/foreign key constraints, normalization, and complex multi-table analytical JOIN queries.
- **Algorithmic Problem Solving Repository (C++ — 225+ Solutions):** Engineered solutions across 5 progressive levels of computational problem solving (matrix operations, string manipulation, combinatorial math, and calendar/date arithmetic engines).
- **Windows Forms Event-Driven Suite (C#, .NET — CSharp-WinForms-Collection):** Developed 10+ desktop applications in C# implementing real-time UI state evaluation, input validation, and asynchronous event handling.